# Proyecto de Grado: IA al servicio de las comunidades

Maestría en Analítica para la Inteligencia de Negocios

**Nombres:** María Fernanda Izquierdo Aparicio & Ana María Ochoa Muñoz

**Fecha:** 19 de Mayo de 2025

Este modelo forma parte del proyecto de grado de la Maestría en Analítica para la Inteligencia de Negocios de la Pontificia Universidad Javeriana, enfocado en la aplicación de Inteligencia Artificial al servicio de las comunidades. Su objetivo es clasificar imágenes de plantas en función de su especie usando una arquitectura pre-entrenada UNet

## 1. Importe de librerias

In [3]:
```
import json

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
from pathlib import Path

import os
import zipfile
import shutil
!pip install seaborn
import seaborn as sns
import datetime
!pip install opencv-python
import cv2


import keras
from keras import models
from keras import layers

import random

import tensorflow as tf
```

```
from tensorflow.keras.models import Model
from tensorflow.keras import Layer
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.preprocessing import image
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing.image import img_to_array, load_img
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, Reduc
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.layers import Conv2D, MaxPooling2D, UpSampling2D, conc

!pip install scikit-learn

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score

from PIL import Image  # Librería para abrir y manipular imágenes

# ignoring warnings
import warnings
warnings.simplefilter("ignore")
```

```
Requirement already satisfied: seaborn in /opt/anaconda3/lib/python3.12/site
-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /opt/anaconda3/lib/py
thon3.12/site-packages (from seaborn) (1.26.4)
Requirement already satisfied: pandas>=1.2 in /opt/anaconda3/lib/python3.12/
site-packages (from seaborn) (2.2.2)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /opt/anaconda3/li
b/python3.12/site-packages (from seaborn) (3.9.2)
Requirement already satisfied: contourpy>=1.0.1 in /opt/anaconda3/lib/python
3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.2.0)
Requirement already satisfied: cycler>=0.10 in /opt/anaconda3/lib/python3.1
2/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in /opt/anaconda3/lib/pytho
n3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /opt/anaconda3/lib/pytho
n3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.4)
Requirement already satisfied: packaging>=20.0 in /opt/anaconda3/lib/python
3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.1)
Requirement already satisfied: pillow>=8 in /opt/anaconda3/lib/python3.12/si
te-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (10.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in /opt/anaconda3/lib/python
3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.1.2)
Requirement already satisfied: python-dateutil>=2.7 in /opt/anaconda3/lib/py
thon3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /opt/anaconda3/lib/python3.1
2/site-packages (from pandas>=1.2->seaborn) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in /opt/anaconda3/lib/python3.
12/site-packages (from pandas>=1.2->seaborn) (2023.3)
Requirement already satisfied: six>=1.5 in /opt/anaconda3/lib/python3.12/sit
e-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.
16.0)
Requirement already satisfied: opencv-python in /opt/anaconda3/lib/python3.1
2/site-packages (4.11.0.86)
Requirement already satisfied: numpy>=1.21.2 in /opt/anaconda3/lib/python3.1
2/site-packages (from opencv-python) (1.26.4)
Requirement already satisfied: scikit-learn in /opt/anaconda3/lib/python3.1
2/site-packages (1.5.1)
Requirement already satisfied: numpy>=1.19.5 in /opt/anaconda3/lib/python3.1
2/site-packages (from scikit-learn) (1.26.4)
Requirement already satisfied: scipy>=1.6.0 in /opt/anaconda3/lib/python3.1
2/site-packages (from scikit-learn) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in /opt/anaconda3/lib/python3.1
2/site-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /opt/anaconda3/lib/py
thon3.12/site-packages (from scikit-learn) (3.5.0)
```

In [4]:

```
#Medir_tiempo_ejecucion():
inicio = datetime.datetime.now()
print(f"Inicio: {inicio.strftime('%Y-%m-%d %H:%M:%S.%f')}")
```

```
Inicio: 2025-05-22 04:36:40.752489
```

# 2. Definción de Rutas y Parámetros

In [8]:
```
data_dir = 'Datos Finales Estado Balanceados'  # Reemplaza con la ruta real
metadata_file = 'archivos_datos_balanceados_estado.csv' # Reemplaza con la r
image_height = 128
image_width = 128
batch_size = 32
num_classes = None  # Se determinará automáticamente del archivo CSV
```

# 3. Cargar y procesar los metadatos

In [11]:
```
try:
    metadata = pd.read_csv(metadata_file, sep=";")
    metadata = metadata.rename(columns={'Nombre del archivo': 'image_id'})
    metadata = metadata.rename(columns={'Planta_Estado': 'Planta_Estado_Espa

    # Asegúrate de que las columnas 'filename' (o el nombre de tu columna de
    # y 'label' (o el nombre de tu columna de etiquetas) existan.

    if 'image_id' not in metadata.columns or 'Planta_Estado_Español' not in
        raise ValueError("El archivo CSV debe contener las columnas 'image_i
    class_names = metadata['Planta_Estado_Español'].unique().tolist()
    num_classes = len(class_names)
    print(f"Se encontraron {num_classes} clases: {class_names}")
except FileNotFoundError:
    print(f"Error: No se encontró el archivo de metadatos en '{metadata_file
    exit()
except ValueError as e:
    print(f"Error al procesar el archivo CSV: {e}")
    exit()
```

```
Se encontraron 18 clases: ['Maíz_Enferma', 'Tomate_Saludable', 'Yuca_Enferm
a', 'Pepino Cohombro_Enferma', 'Tomate_Enferma', 'Manzana_Saludable', 'Fresa
_Enferma', 'Maíz_Saludable', 'Papa_Enferma', 'Fresa_Saludable', 'Coliflor_En
ferma', 'Pepino Cohombro_Saludable', 'Yuca_Saludable', 'Fríjol_Enferma', 'Pa
pa_Saludable', 'Fríjol_Saludable', 'Coliflor_Saludable', 'Manzana_Enferma']
```

In [13]:
```
metadata.Planta_Estado_Español.value_counts()
```

Out[13]:
```
Planta_Estado_Español
Fríjol_Saludable            3501
Maíz_Enferma                3500
Tomate_Saludable            3500
Coliflor_Saludable          3500
Papa_Saludable              3500
Fríjol_Enferma              3500
Yuca_Saludable              3500
Pepino Cohombro_Saludable   3500
Coliflor_Enferma            3500
Fresa_Saludable             3500
Papa_Enferma                3500
Maíz_Saludable              3500
Fresa_Enferma               3500
Manzana_Saludable           3500
Tomate_Enferma              3500
Pepino Cohombro_Enferma     3500
Yuca_Enferma                3500
Manzana_Enferma             3500
Name: count, dtype: int64
```

In [15]:
```
train, test= train_test_split(metadata, test_size=0.2, random_state=42,shuff
train.shape, test.shape
```

Out[15]:
```
((50400, 7), (12601, 7))
```

In [17]:
```
train.head()
```

Out[17]:

| | Carpeta | image_id | Planta_Inglés | Estado_Ingles |
|---|---|---|---|---|
| **50219** | Potato_healthy | aug_966_683b04ad-7941-4819-9965-ba32c725eb22___... | Potato | healthy |
| **12719** | Cucumber_sick | aug_60_cucumber_photo_2020-06-27_23-51-02 (2).jpg | Cucumber | sick |
| **23065** | Strawberry_sick | aug_124_36bc7257-0b59-45a9-a692-1ff35b5a3425___... | Strawberry | sick |
| **42154** | Cassava_healthy | aug_799_919479322.jpg | Cassava | healthy |
| **60493** | Apple_sick | b1a2c1e8-e1a6-4d25-9d5b-1488e9d8b0ee___FREC_C.... | Apple | sick |

In [19]:
```
sample = train.sample(5)  # ejemplo común
print(sample)
```

```
               Carpeta                                         image_id  \
15760      Tomato_sick  ffbb4b17-6760-48fd-bbfa-70120a53db86___Matt.S_...
12216    Cucumber_sick    aug_1196_cucumber_photo_2020-06-27_23-54-54.jpg
16216      Tomato_sick  ab09b502-9c17-4764-bdea-31c128389c40___YLCV_NR...
49268  Potato_healthy  aug_3063_03da9931-e514-4cc7-b04a-8f474a133ce5_...
29741      Potato_sick  c206ebe3-c06e-4eb9-a537-a719cfa1e3c0___RS_LB 5...

      Planta_Inglés Estado_Ingles   Planta_Español Estado_Español  \
15760        Tomato          sick           Tomate        Enferma
12216      Cucumber          sick  Pepino Cohombro        Enferma
16216        Tomato          sick           Tomate        Enferma
49268        Potato       healthy             Papa      Saludable
29741        Potato          sick             Papa        Enferma

         Planta_Estado_Español
15760           Tomate_Enferma
12216  Pepino Cohombro_Enferma
16216           Tomate_Enferma
49268           Papa_Saludable
29741             Papa_Enferma
```

In [21]:
```
print(sample.columns)
```

```
Index(['Carpeta', 'image_id', 'Planta_Inglés', 'Estado_Ingles',
       'Planta_Español', 'Estado_Español', 'Planta_Estado_Español'],
      dtype='object')
```

# 4. Crear un generador de datos

In [24]:
```
df_subset = train.sample(frac=0.50, random_state=42)  # Solo 50% de los datc
df_subset.head()
```

Out[24]:

| | Carpeta | image_id | Planta_Inglés | Estado_In |
|---|---|---|---|---|
| **1746** | Corn_sick | bc43c710-9fa2-44a2-8420-94c71c01c2c6___RS_GLSp... | Corn | |
| **30848** | Potato_sick | aug_770_31d5d932-9eb3-45d1-bc85-40ef64d7a688___... | Potato | |
| **15267** | Tomato_sick | ed85b79f-958f-42a8-adb9-2a34022fa016___Crnl_L.... | Tomato | |
| **21171** | Strawberry_sick | aug_572_2f54dce1-5aa1-43fb-8eb4-8aeb18419440___... | Strawberry | |
| **48779** | Bean_sick | aug_1732_bean_bean_rust_train.278.jpg | Bean | |

In [26]:
```
df_subset.shape
```

Out[26]: (25200, 7)

In [28]:
```
# Crear el generador de imágenes
datagen = ImageDataGenerator(
    validation_split = 0.2,
    rescale=1./255
)

train_generator = datagen.flow_from_dataframe(
    dataframe=df_subset,
    directory=data_dir,
    x_col='image_id',
    y_col='Planta_Estado_Español',
    target_size=(image_height, image_width),
    batch_size=batch_size,
    class_mode='categorical',  # Importante para clasificación multiclase
    subset='training'
)

validation_generator = datagen.flow_from_dataframe(
    dataframe=df_subset,
    directory=data_dir,
    x_col='image_id',
    y_col='Planta_Estado_Español',
    target_size=(image_height, image_width),
    batch_size=batch_size,
    class_mode='categorical',
    subset='validation'
)
```

```
Found 20160 validated image filenames belonging to 18 classes.
Found 5039 validated image filenames belonging to 18 classes.
```

In [29]:
```
print(train['Planta_Estado_Español'].value_counts())
print(f"Total clases: {train['Planta_Estado_Español'].nunique()}")
```

```
Planta_Estado_Español
Papa_Saludable               2800
Pepino Cohombro_Enferma      2800
Tomate_Saludable             2800
Papa_Enferma                 2800
Maíz_Saludable               2800
Fríjol_Enferma               2800
Coliflor_Enferma             2800
Pepino Cohombro_Saludable    2800
Fríjol_Saludable             2800
Manzana_Saludable            2800
Coliflor_Saludable           2800
Yuca_Enferma                 2800
Tomate_Enferma               2800
Fresa_Saludable              2800
Manzana_Enferma              2800
Yuca_Saludable               2800
Fresa_Enferma                2800
Maíz_Enferma                 2800
Name: count, dtype: int64
Total clases: 18
```

In [32]:

```python
from tensorflow.keras.preprocessing import image
import os
import numpy as np

img_path = os.path.join(data_dir, metadata["image_id"][20])
img = image.load_img(img_path, target_size=(image_height, image_width))
img_tensor = image.img_to_array(img)
img_tensor = np.expand_dims(img_tensor, axis=0)
img_tensor /= 255.

plt.imshow(img_tensor[0])
plt.axis('off')
plt.show()
```

# 5. Definición del modelo U-Net para clasificación

In [35]:
```
epochs=10
steps_per_epoch = len(train)*0.8 / batch_size
validation_steps = len(train)*0.2 / batch_size
```

In [37]:
```
# DEFINIR EL MODELO U-NET MODIFICADO PARA CLASIFICACIÓN
def unet_classification_model(input_shape=(image_height, image_width, 3), nu
    inputs = Input(shape=input_shape)

    # Encoder
    c1 = Conv2D(32, 3, activation='relu', padding='same')(inputs)
    c1 = Conv2D(32, 3, activation='relu', padding='same')(c1)
    p1 = MaxPooling2D()(c1)

    c2 = Conv2D(64, 3, activation='relu', padding='same')(p1)
    c2 = Conv2D(64, 3, activation='relu', padding='same')(c2)
    p2 = MaxPooling2D()(c2)

    c3 = Conv2D(128, 3, activation='relu', padding='same')(p2)
    c3 = Conv2D(128, 3, activation='relu', padding='same')(c3)
    p3 = MaxPooling2D()(c3)

    # Bottleneck
```

```
    c4 = Conv2D(256, 3, activation='relu', padding='same')(p3)
    c4 = Conv2D(256, 3, activation='relu', padding='same')(c4)

    # Decoder
    u5 = UpSampling2D()(c4)
    u5 = concatenate([u5, c3])
    c5 = Conv2D(128, 3, activation='relu', padding='same')(u5)
    c5 = Conv2D(128, 3, activation='relu', padding='same')(c5)

    u6 = UpSampling2D()(c5)
    u6 = concatenate([u6, c2])
    c6 = Conv2D(64, 3, activation='relu', padding='same')(u6)
    c6 = Conv2D(64, 3, activation='relu', padding='same')(c6)

    u7 = UpSampling2D()(c6)
    u7 = concatenate([u7, c1])
    c7 = Conv2D(32, 3, activation='relu', padding='same')(u7)
    c7 = Conv2D(32, 3, activation='relu', padding='same')(c7)

    # Clasificación
    gap = GlobalAveragePooling2D()(c7)
    outputs = Dense(num_classes, activation='softmax')(gap)

    model = Model(inputs=inputs, outputs=outputs)
    return model


# CREAR Y COMPILAR EL MODELO
model = unet_classification_model(input_shape=(image_height, image_width, 3)
model.compile(optimizer=Adam(), loss='categorical_crossentropy', metrics=['a
model.summary()  # Muestra la arquitectura
```

**Model: "functional"**

| Layer (type) | Output Shape | Param # | Connected to |
|---|---|---|---|
| input_layer (InputLayer) | (None, 128, 128, 3) | 0 | - |
| conv2d (Conv2D) | (None, 128, 128, 32) | 896 | input_layer[0 |
| conv2d_1 (Conv2D) | (None, 128, 128, 32) | 9,248 | conv2d[0][0] |
| max_pooling2d (MaxPooling2D) | (None, 64, 64, 32) | 0 | conv2d_1[0][0 |
| conv2d_2 (Conv2D) | (None, 64, 64, 64) | 18,496 | max_pooling2d |

| | | | |
|---|---|---|---|
| conv2d_3 (Conv2D) | (None, 64, 64, 64) | 36,928 | conv2d_2[0][0 |
| max_pooling2d_1 (MaxPooling2D) | (None, 32, 32, 64) | 0 | conv2d_3[0][0 |
| conv2d_4 (Conv2D) | (None, 32, 32, 128) | 73,856 | max_pooling2d |
| conv2d_5 (Conv2D) | (None, 32, 32, 128) | 147,584 | conv2d_4[0][0 |
| max_pooling2d_2 (MaxPooling2D) | (None, 16, 16, 128) | 0 | conv2d_5[0][0 |
| conv2d_6 (Conv2D) | (None, 16, 16, 256) | 295,168 | max_pooling2d |
| conv2d_7 (Conv2D) | (None, 16, 16, 256) | 590,080 | conv2d_6[0][0 |
| up_sampling2d (UpSampling2D) | (None, 32, 32, 256) | 0 | conv2d_7[0][0 |
| concatenate (Concatenate) | (None, 32, 32, 384) | 0 | up_sampling2d conv2d_5[0][0 |
| conv2d_8 (Conv2D) | (None, 32, 32, 128) | 442,496 | concatenate[0 |
| conv2d_9 (Conv2D) | (None, 32, 32, 128) | 147,584 | conv2d_8[0][0 |
| up_sampling2d_1 (UpSampling2D) | (None, 64, 64, 128) | 0 | conv2d_9[0][0 |
| concatenate_1 (Concatenate) | (None, 64, 64, 192) | 0 | up_sampling2d conv2d_3[0][0 |
| conv2d_10 (Conv2D) | (None, 64, 64, 64) | 110,656 | concatenate_1 |
| conv2d_11 (Conv2D) | (None, 64, 64, 64) | 36,928 | conv2d_10[0][ |
| up_sampling2d_2 (UpSampling2D) | (None, 128, 128, 64) | 0 | conv2d_11[0][ |
| concatenate_2 (Concatenate) | (None, 128, 128, 96) | 0 | up_sampling2d conv2d_1[0][0 |
| conv2d_12 (Conv2D) | (None, 128, 128, 32) | 27,680 | concatenate_2 |

| | | | |
|---|---|---|---|
| conv2d_13 (Conv2D) | (None, 128, 128, 32) | 9,248 | conv2d_12[0][ |
| global_average_poo… (GlobalAveragePool… | (None, 32) | 0 | conv2d_13[0][ |
| dense (Dense) | (None, 18) | 594 | global_averag |

**Total params:** 1,947,442 (7.43 MB)

**Trainable params:** 1,947,442 (7.43 MB)

**Non-trainable params:** 0 (0.00 B)

In [39]:
```
# Earlystopping
from tensorflow.keras.callbacks import EarlyStopping

early_stop = EarlyStopping(
    monitor='val_loss',      # Monitorea la pérdida en validación
    patience=5,              # Número de épocas sin mejora antes de detener
    restore_best_weights=True,  # Restaura los mejores pesos
    verbose=1
)
```

# 6. Entrenamiento del Modelo

In [42]:
```
#Entrenamiento
history = model.fit(
    train_generator,
    validation_data=validation_generator,
    epochs=epochs,
    steps_per_epoch=int(steps_per_epoch),
    validation_steps=int(validation_steps),
    callbacks=[early_stop]
)
```

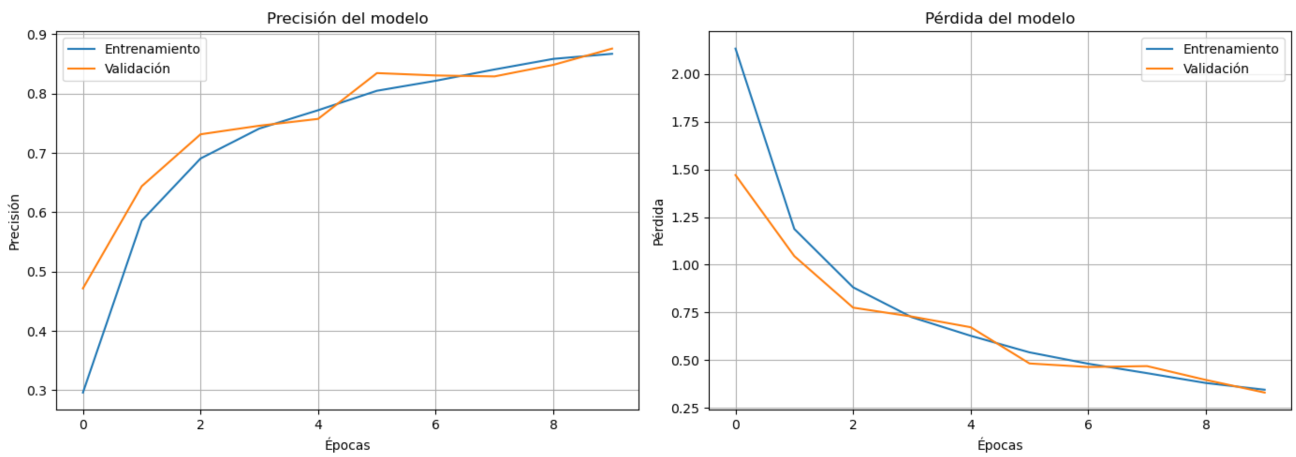
```
Epoch 1/10
1260/1260 ━━━━━━━━━━━━━━━━━━━━ 1751s 1s/step - accuracy: 0.2340 - loss: 2.35
07 - val_accuracy: 0.4717 - val_loss: 1.4702
Epoch 2/10
1260/1260 ━━━━━━━━━━━━━━━━━━━━ 1769s 1s/step - accuracy: 0.5682 - loss: 1.24
26 - val_accuracy: 0.6440 - val_loss: 1.0456
Epoch 3/10
1260/1260 ━━━━━━━━━━━━━━━━━━━━ 1785s 1s/step - accuracy: 0.6821 - loss: 0.91
00 - val_accuracy: 0.7315 - val_loss: 0.7754
Epoch 4/10
1260/1260 ━━━━━━━━━━━━━━━━━━━━ 1804s 1s/step - accuracy: 0.7396 - loss: 0.73
05 - val_accuracy: 0.7460 - val_loss: 0.7286
Epoch 5/10
1260/1260 ━━━━━━━━━━━━━━━━━━━━ 2984s 2s/step - accuracy: 0.7695 - loss: 0.63
76 - val_accuracy: 0.7575 - val_loss: 0.6727
Epoch 6/10
1260/1260 ━━━━━━━━━━━━━━━━━━━━ 1800s 1s/step - accuracy: 0.8036 - loss: 0.54
88 - val_accuracy: 0.8347 - val_loss: 0.4829
Epoch 7/10
1260/1260 ━━━━━━━━━━━━━━━━━━━━ 1802s 1s/step - accuracy: 0.8222 - loss: 0.48
23 - val_accuracy: 0.8307 - val_loss: 0.4642
Epoch 8/10
1260/1260 ━━━━━━━━━━━━━━━━━━━━ 1831s 1s/step - accuracy: 0.8359 - loss: 0.44
65 - val_accuracy: 0.8291 - val_loss: 0.4689
Epoch 9/10
1260/1260 ━━━━━━━━━━━━━━━━━━━━ 1985s 2s/step - accuracy: 0.8545 - loss: 0.39
08 - val_accuracy: 0.8486 - val_loss: 0.3965
Epoch 10/10
1260/1260 ━━━━━━━━━━━━━━━━━━━━ 1945s 2s/step - accuracy: 0.8673 - loss: 0.34
08 - val_accuracy: 0.8760 - val_loss: 0.3303
Restoring model weights from the end of the best epoch: 10.
```

In [43]:
```
fin = datetime.datetime.now()
print(f"Fin: {fin.strftime('%Y-%m-%d %H:%M:%S.%f')}")

duracion = fin - inicio
print(f"Duración: {duracion}")
```

```
Fin: 2025-05-22 10:01:48.769628
Duración: 5:25:08.017139
```

In [44]:
```
import matplotlib.pyplot as plt

# 1. Crear la figura
plt.figure(figsize=(14, 5))

# 2. Gráfico de Accuracy
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Entrenamiento')
plt.plot(history.history['val_accuracy'], label='Validación')
```

```
plt.title('Precisión del modelo')
plt.xlabel('Épocas')
plt.ylabel('Precisión')
plt.legend()
plt.grid(True)

# 3. Gráfico de Loss
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Entrenamiento')
plt.plot(history.history['val_loss'], label='Validación')
plt.title('Pérdida del modelo')
plt.xlabel('Épocas')
plt.ylabel('Pérdida')
plt.legend()
plt.grid(True)

# 4. Mostrar
plt.tight_layout()
plt.show()
```




In [45]:
```
print(test.columns.tolist())
```

```
['Carpeta', 'image_id', 'Planta_Inglés', 'Estado_Ingles', 'Planta_Español',
'Estado_Español', 'Planta_Estado_Español']
```

In [46]:
```
epochs = list(range(1, 11))

loss = history.history['loss']
val_loss = history.history['val_loss']
accuracy = history.history['accuracy']
val_accuracy = history.history['val_accuracy']


data = {
    'Epoch': epochs,
    'Trainning Loss': loss,
    'Validation Loss': val_loss,
```

```
    'Training Accuracy': accuracy,
    'Validation  Accuracy': val_accuracy
}
df = pd.DataFrame(data)

# Imprimir la tabla
print(df)
```

```
   Epoch  Trainning Loss  Validation Loss  Training Accuracy  \
0      1        2.133118         1.470182           0.296081   
1      2        1.187853         1.045571           0.585962   
2      3        0.881940         0.775419           0.690675   
3      4        0.725062         0.728633           0.741220   
4      5        0.627924         0.672681           0.772123   
5      6        0.541022         0.482889           0.804911   
6      7        0.481430         0.464173           0.821776   
7      8        0.431966         0.468946           0.840823   
8      9        0.380362         0.396479           0.858631   
9     10        0.345180         0.330347           0.867212   

   Validation  Accuracy  
0              0.471721  
1              0.643977  
2              0.731494  
3              0.745981  
4              0.757492  
5              0.834689  
6              0.830720  
7              0.829133  
8              0.848581  
9              0.875967  
```

In [47]:
```
print('Perdida final en Train:', history.history['loss'][-1])
print("Pérdida final en Validation:", history.history['val_loss'][-1])

print("Accuracy final en Train:", history.history['accuracy'][-1])
print("Accuracy final en Validation:", history.history['val_accuracy'][-1])
```

```
Perdida final en Train: 0.3451800048351288
Pérdida final en Validation: 0.3303467929363251
Accuracy final en Train: 0.8672122955322266
Accuracy final en Validation: 0.8759674429893494
```

In [48]:
```
#validación con datos de test

test_generator = datagen.flow_from_dataframe(test,
                         directory = "Datos Finales",
                         x_col = "image_id",
                         y_col = "Planta_Estado_Español",
                         target_size = (image_height,image_width),
                         batch_size = 1,
```

```
                                    shuffle=False,
                                    class_mode = "categorical")
```

```
Found 4903 validated image filenames belonging to 18 classes.
```

# 7. Testeo del Modelo

In [50]:

```
# Evaluar sobre el set de test
test_loss, test_accuracy = model.evaluate(test_generator, steps=len(test_ger

print(f'🔎 Pérdida en test: {test_loss:.4f}')
print(f'✅ Precisión en test: {test_accuracy:.2%}')
```

```
4903/4903 ━━━━━━━━━━━━━━━━━━━━ 159s 32ms/step - accuracy: 0.8244 - loss: 0.4
756
🔎 Pérdida en test: 0.4771
✅ Precisión en test: 82.05%
```

In [51]:

```
# 1. Predecir clases sobre el test set
y_pred_probs = model.predict(test_generator)

# Convertir probabilidades a etiquetas (argmax)
y_pred = np.argmax(y_pred_probs, axis=1)

# 2. Obtener etiquetas verdaderas
y_true = test_generator.classes
```

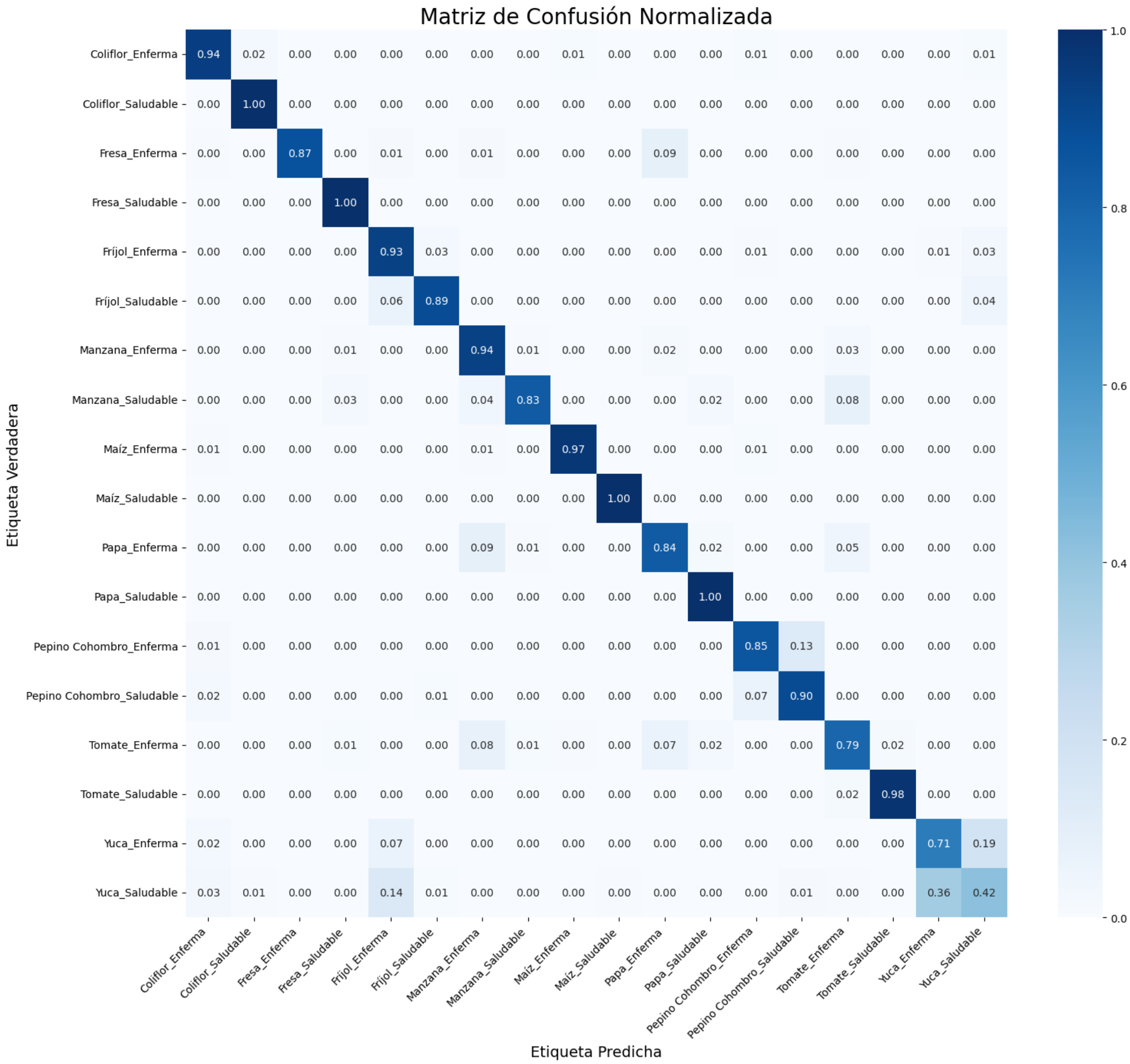
```
4903/4903 ━━━━━━━━━━━━━━━━━━━━ 159s 32ms/step
```

In [ ]:

In [53]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

# 1. Predecir clases sobre el test set
# Obtener las predicciones del modelo
#y_pred_probs = model.predict(test_generator, steps=len(test_generator), ver

# Convertir probabilidades a etiquetas (argmax)
#_pred = np.argmax(y_pred_probs, axis=1)

# 2. Obtener etiquetas verdaderas
# Las verdaderas etiquetas están en test_generator.classes
#y_true = test_generator.classes

# 3. Obtener los nombres de las clases
```

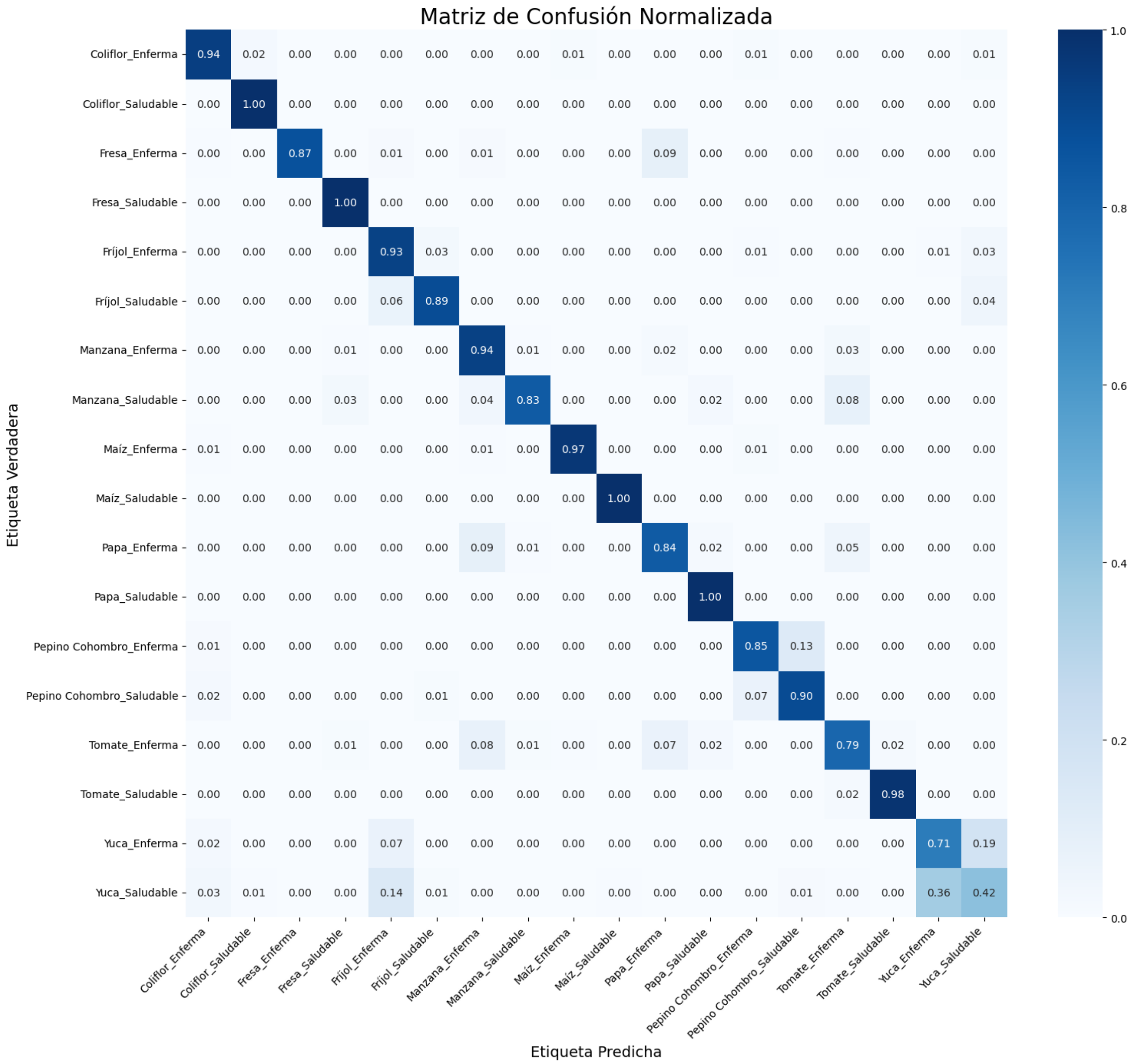
```
class_labels = list(test_generator.class_indices.keys())

# 4. Calcular la matriz de confusión
cm = confusion_matrix(y_true, y_pred)

# 5. Normalizar la matriz de confusión (opcional para mejor visualización)
cm_normalized = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]

# 6. Graficar la matriz de confusión
plt.figure(figsize=(16, 14))
sns.heatmap(cm_normalized, annot=True, fmt='.2f', cmap='Blues', xticklabels=
plt.title('Matriz de Confusión Normalizada', fontsize=20)
plt.ylabel('Etiqueta Verdadera', fontsize=14)
plt.xlabel('Etiqueta Predicha', fontsize=14)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

# 7. Reporte de clasificación (precisión, recall, f1-score por clase)
print('\nReporte de Clasificación:')
print(classification_report(y_true, y_pred, target_names=class_labels, digit
```


Matriz de Confusión Normalizada
Etiqueta Verdadera
Etiqueta Predicha
Coliflor_Enferma 0.94 0.02 0.00 0.00 0.00 0.00 0.00 0.00 0.01 0.00 0.00 0.00 0.01 0.00 0.00 0.00 0.00 0.01
Coliflor_Saludable 0.00 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
Fresa_Enferma 0.00 0.00 0.87 0.00 0.01 0.00 0.01 0.00 0.00 0.00 0.09 0.00 0.00 0.00 0.00 0.00 0.00 0.00
Fresa_Saludable 0.00 0.00 0.00 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
Frijol_Enferma 0.00 0.00 0.00 0.00 0.93 0.03 0.00 0.00 0.00 0.00 0.00 0.00 0.01 0.00 0.00 0.00 0.01 0.03
Fríjol_Saludable 0.00 0.00 0.00 0.00 0.06 0.89 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.04
Manzana_Enferma 0.00 0.00 0.00 0.01 0.00 0.00 0.94 0.01 0.00 0.00 0.02 0.00 0.00 0.00 0.03 0.00 0.00 0.00
Manzana_Saludable 0.00 0.00 0.00 0.03 0.00 0.00 0.04 0.83 0.00 0.00 0.00 0.02 0.00 0.00 0.08 0.00 0.00 0.00
Maíz_Enferma 0.01 0.00 0.00 0.00 0.00 0.00 0.01 0.00 0.97 0.00 0.00 0.00 0.01 0.00 0.00 0.00 0.00 0.00
Maíz_Saludable 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
Papa_Enferma 0.00 0.00 0.00 0.00 0.00 0.00 0.09 0.01 0.00 0.00 0.84 0.02 0.00 0.00 0.05 0.00 0.00 0.00
Papa_Saludable 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 1.00 0.00 0.00 0.00 0.00 0.00 0.00
Pepino Cohombro_Enferma 0.01 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.85 0.13 0.00 0.00 0.00 0.00
Pepino Cohombro_Saludable 0.02 0.00 0.00 0.00 0.00 0.01 0.00 0.00 0.00 0.00 0.00 0.00 0.07 0.90 0.00 0.00 0.00 0.00
Tomate_Enferma 0.00 0.00 0.00 0.01 0.00 0.00 0.08 0.01 0.00 0.00 0.07 0.02 0.00 0.00 0.79 0.02 0.00 0.00
Tomate_Saludable 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.02 0.98 0.00 0.00
Yuca_Enferma 0.02 0.00 0.00 0.00 0.07 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.71 0.19
Yuca_Saludable 0.03 0.01 0.00 0.00 0.14 0.01 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.01 0.00 0.00 0.36 0.42
Coliflor_Enferma
Coliflor_Saludable
Fresa_Enferma
Fresa_Saludable
Fríjol_Enferma
Fríjol_Saludable
Manzana_Enferma
Manzana_Saludable
Maíz_Enferma
Maíz_Saludable
Papa_Enferma
Papa_Saludable
Pepino Cohombro_Enferma
Pepino Cohombro_Saludable
Tomate_Enferma
Tomate_Saludable
Yuca_Enferma
Yuca_Saludable
1.0
0.8
0.6
0.4
0.2
0.0

```
Reporte de Clasificación:
                            precision    recall  f1-score   support

          Coliflor_Enferma     0.6803    0.9432    0.7905        88
        Coliflor_Saludable     0.7805    1.0000    0.8767        32
             Fresa_Enferma     0.9947    0.8744    0.9307       215
           Fresa_Saludable     0.8349    1.0000    0.9100        91
            Fríjol_Enferma     0.5265    0.9313    0.6727       160
          Fríjol_Saludable     0.8737    0.8925    0.8830        93
           Manzana_Enferma     0.7206    0.9356    0.8142       295
         Manzana_Saludable     0.9626    0.8324    0.8927       340
              Maíz_Enferma     0.9907    0.9708    0.9806       548
            Maíz_Saludable     0.9916    1.0000    0.9958       236
              Papa_Enferma     0.8168    0.8354    0.8260       395
            Papa_Saludable     0.5000    1.0000    0.6667        29
    Pepino Cohombro_Enferma     0.8056    0.8529    0.8286        68
  Pepino Cohombro_Saludable     0.8105    0.8953    0.8508        86
            Tomate_Enferma     0.8948    0.7900    0.8392       700
          Tomate_Saludable     0.9543    0.9812    0.9675       319
              Yuca_Enferma     0.7283    0.7086    0.7183       700
            Yuca_Saludable     0.6011    0.4213    0.4954       508

                  accuracy                         0.8205      4903
                 macro avg     0.8038    0.8814    0.8300      4903
              weighted avg     0.8290    0.8205    0.8183      4903
```

In [54]:

```python
# 1. Guardar la figura de la matriz de confusión
plt.figure(figsize=(16, 14))
sns.heatmap(cm_normalized, annot=True, fmt='.2f', cmap='Blues', xticklabels=
plt.title('Matriz de Confusión Normalizada', fontsize=20)
plt.ylabel('Etiqueta Verdadera', fontsize=14)
plt.xlabel('Etiqueta Predicha', fontsize=14)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()

# Guardar el gráfico como imagen
plt.savefig(r'C:\Proyecto de grado\Modelos\matriz_confusion.png', dpi=300)
plt.show()
```




In [55]:
```
# 2. Guardar el reporte de clasificación en CSV

# Convertir el reporte a un diccionario
from sklearn.metrics import classification_report

report = classification_report(y_true, y_pred, target_names=class_labels, ou

# Convertirlo a DataFrame
report_df = pd.DataFrame(report).transpose()

# Guardar como CSV
report_df.to_csv('reporte_clasificacion_modelo16_UNet_Planta_Estado_sin_Data

print("✅ Reporte de clasificación guardado")
```

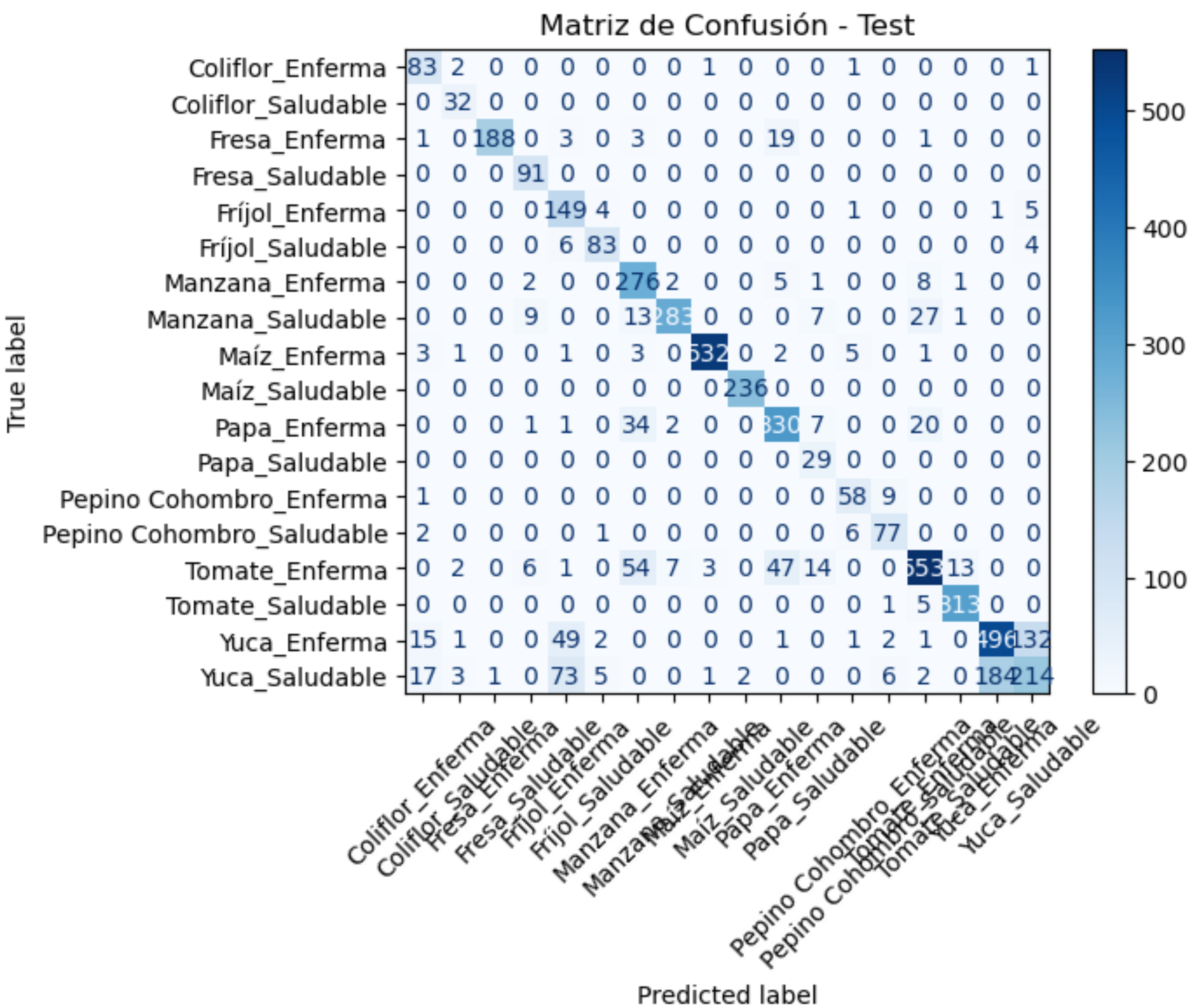
✅ Reporte de clasificación guardado

In [56]:
```
accuracy = accuracy_score(y_true, y_pred)
print(f"Test Accuracy: {accuracy:.4f}")
```

Test Accuracy: 0.8205

In [ ]:

In [58]:
```
# --------------------------
# 2. Matriz de Confusión
# --------------------------
from sklearn.metrics import confusion_matrix, accuracy_score, ConfusionMatri

cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=test_gener
disp.plot(cmap="Blues", xticks_rotation=45)
plt.title("Matriz de Confusión - Test")
plt.show()
```



In [ ]:

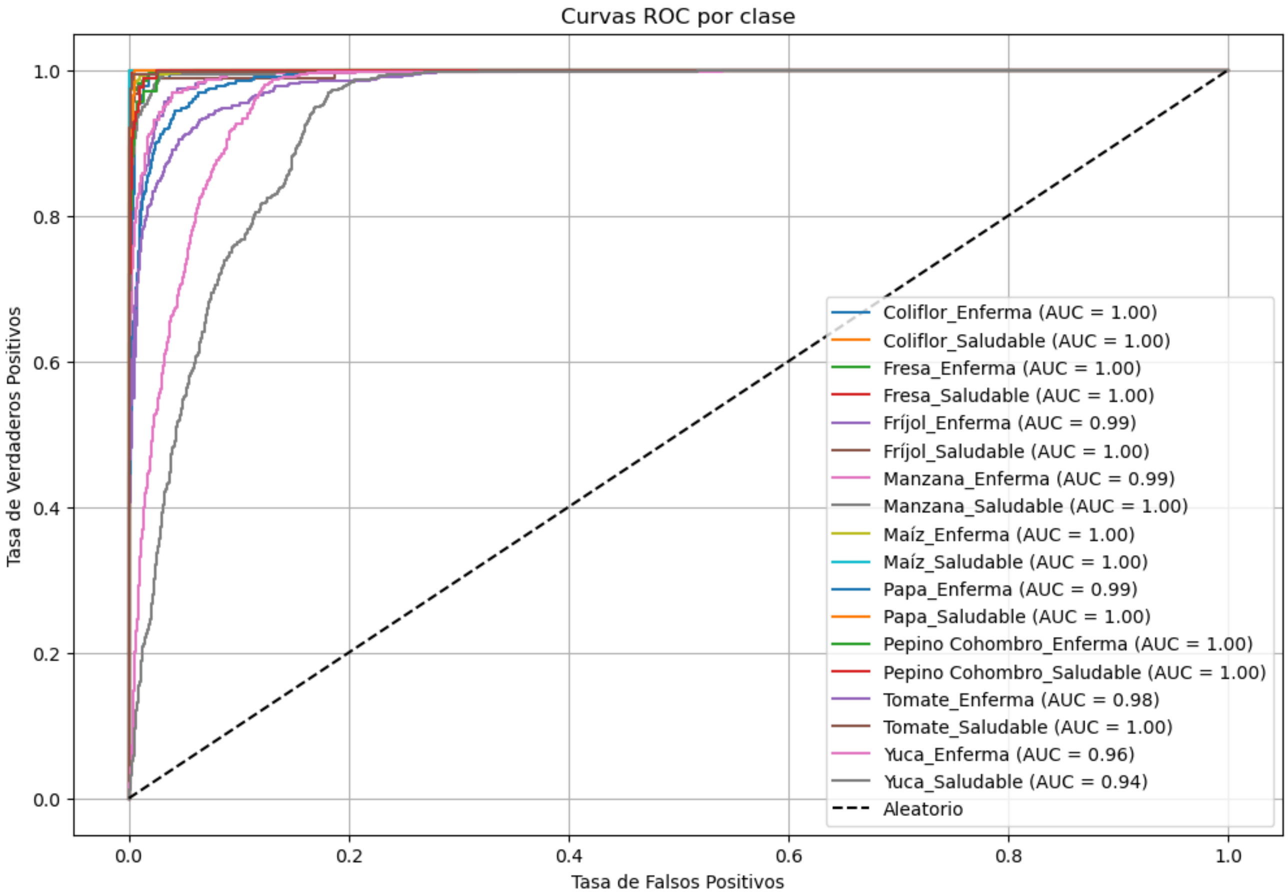
In [60]:
```
# Función para normalizar imágenes solo para visualización
def normalize_for_display(img):
    img_min = img.min()
    img_max = img.max()
    if img_max > 1.0 or img_min < 0.0:
        img = (img - img_min) / (img_max - img_min + 1e-7)
    return img

# Obtener nombres de clases
class_indices = test_generator.class_indices
class_names = list(class_indices.keys())

# Reiniciar el generador para empezar desde el principio
test_generator.reset()
x_batch, y_true_batch = next(test_generator)

# Obtener predicciones del modelo
y_pred_probs_batch = model.predict(x_batch)
y_pred_batch = np.argmax(y_pred_probs_batch, axis=1)
y_true_batch = np.argmax(y_true_batch, axis=1)

# Número de imágenes a mostrar (máximo el tamaño del batch)
num_samples = x_batch.shape[0]

# Mostrar imágenes con etiquetas
plt.figure(figsize=(15, 8))
for i in range(num_samples):
    plt.subplot(3, 4, i + 1)
    img_to_show = normalize_for_display(x_batch[i])
    plt.imshow(img_to_show)
    plt.axis('off')
    plt.title(
        f"Real: {class_names[y_true_batch[i]]}\nPred: {class_names[y_pred_ba
        color='green' if y_true_batch[i] == y_pred_batch[i] else 'red'
    )
plt.tight_layout()
plt.suptitle("Muestreo de Predicciones vs. Etiquetas Reales", fontsize=16, y
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 130ms/step
```

Muestreo de Predicciones vs. Etiquetas Reales

Real: Yuca_Enferma
Pred: Yuca_Enferma

In [61]:
```
from sklearn.metrics import roc_curve, auc
from tensorflow.keras.utils import to_categorical

# 3. Convertir etiquetas reales a one-hot
n_classes = y_pred_probs.shape[1]
y_true_onehot = to_categorical(y_true, num_classes=n_classes)

# 4. Calcular ROC y AUC por clase
fpr = dict()
tpr = dict()
roc_auc = dict()

for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_true_onehot[:, i], y_pred_probs[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# 5. Obtener nombres de clase
class_names = list(test_generator.class_indices.keys())

# 6. Graficar curvas ROC
plt.figure(figsize=(12, 8))
for i in range(n_classes):
    plt.plot(fpr[i], tpr[i], label=f'{class_names[i]} (AUC = {roc_auc[i]:.2f

plt.plot([0, 1], [0, 1], 'k--', label='Aleatorio')
plt.xlabel('Tasa de Falsos Positivos')
plt.ylabel('Tasa de Verdaderos Positivos')
plt.title('Curvas ROC por clase')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```




In [62]:
```
from sklearn.metrics import roc_auc_score

# AUC macro (promedio de clases, todas igual de importantes)
auc_macro = roc_auc_score(y_true_onehot, y_pred_probs, average='macro')

# AUC micro (promedia sobre todas las instancias)
auc_micro = roc_auc_score(y_true_onehot, y_pred_probs, average='micro')

print(f"AUC Macro promedio: {auc_macro:.4f}")
print(f"AUC Micro promedio: {auc_micro:.4f}")
```

```
AUC Macro promedio: 0.9914
AUC Micro promedio: 0.9932
```

In [63]:
```
# Asegúrate de tener esto:
# y_pred = np.argmax(y_pred_probs, axis=1)
# y_true = test_generator.classes

# Accuracy
accuracy = accuracy_score(y_true, y_pred)

# F1 Scores
f1_macro = f1_score(y_true, y_pred, average='macro')
f1_micro = f1_score(y_true, y_pred, average='micro')
```

```python
f1_weighted = f1_score(y_true, y_pred, average='weighted')

print(f"Accuracy: {accuracy:.4f}")
print(f"F1 Score (macro): {f1_macro:.4f}")
print(f"F1 Score (micro): {f1_micro:.4f}")
print(f"F1 Score (weighted): {f1_weighted:.4f}")

# Detalle por clase
class_names = list(test_generator.class_indices.keys())
print("\nReporte de clasificación por clase:")
print(classification_report(y_true, y_pred, target_names=class_names))
```

```
Accuracy: 0.8205
F1 Score (macro): 0.8300
F1 Score (micro): 0.8205
F1 Score (weighted): 0.8183

Reporte de clasificación por clase:
                           precision    recall  f1-score   support

         Coliflor_Enferma       0.68      0.94      0.79        88
       Coliflor_Saludable       0.78      1.00      0.88        32
            Fresa_Enferma       0.99      0.87      0.93       215
          Fresa_Saludable       0.83      1.00      0.91        91
           Fríjol_Enferma       0.53      0.93      0.67       160
         Fríjol_Saludable       0.87      0.89      0.88        93
          Manzana_Enferma       0.72      0.94      0.81       295
        Manzana_Saludable       0.96      0.83      0.89       340
             Maíz_Enferma       0.99      0.97      0.98       548
           Maíz_Saludable       0.99      1.00      1.00       236
             Papa_Enferma       0.82      0.84      0.83       395
           Papa_Saludable       0.50      1.00      0.67        29
  Pepino Cohombro_Enferma       0.81      0.85      0.83        68
Pepino Cohombro_Saludable       0.81      0.90      0.85        86
           Tomate_Enferma       0.89      0.79      0.84       700
         Tomate_Saludable       0.95      0.98      0.97       319
             Yuca_Enferma       0.73      0.71      0.72       700
           Yuca_Saludable       0.60      0.42      0.50       508

                 accuracy                           0.82      4903
                macro avg       0.80      0.88      0.83      4903
             weighted avg       0.83      0.82      0.82      4903
```

# 8. Guardar el modelo entrenado

In [64]:
```python
model.save('modelo_16_Unet_PlantaEstado_Balanceado_Sin_Data_Aug_50perc.h5')
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
```

In [65]:

```
fin = datetime.datetime.now()
print(f"Fin: {fin.strftime('%Y-%m-%d %H:%M:%S.%f')}")

duracion = fin - inicio
print(f"Duración: {duracion}")
```

```
Fin: 2025-05-22 10:07:11.746021
Duración: 5:30:30.993532
```

In [66]:

```
class_indices = train_generator.class_indices  # o test_generator.class_indi
index_to_class = {v: k for k, v in class_indices.items()}
print(index_to_class)
with open("index_to_class_modelo_16_Unet_PlantaEstado_Balanceado_Sin_Data_Au
    json.dump(index_to_class, f, indent=4, ensure_ascii=False)
```

```
{0: 'Coliflor_Enferma', 1: 'Coliflor_Saludable', 2: 'Fresa_Enferma', 3: 'Fresa_Saludable', 4: 'Fríjol_Enferma', 5: 'Fríjol_Saludable', 6: 'Manzana_Enferma', 7: 'Manzana_Saludable', 8: 'Maíz_Enferma', 9: 'Maíz_Saludable', 10: 'Papa_Enferma', 11: 'Papa_Saludable', 12: 'Pepino Cohombro_Enferma', 13: 'Pepino Cohombro_Saludable', 14: 'Tomate_Enferma', 15: 'Tomate_Saludable', 16: 'Yuca_Enferma', 17: 'Yuca_Saludable'}
```

In [ ]:

In [ ]:

In [ ]: